

Distinct Subsequences II

Personal DSA Notes · Counting distinct non-empty subsequences of a string

- DYNAMIC PROGRAMMING
- STRING
- LAST OCCURRENCE TRICK
- O(N) TIME

1 Problem

Given a string `s`, count the number of **distinct non-empty subsequences** of `s`. Since the count can be large, return it modulo $10^9 + 7$.

A subsequence is formed by deleting zero or more characters from `s` without changing the relative order of the remaining characters. The key difficulty is that the same subsequence string can be generated in multiple ways when characters repeat, so those repeats must be counted only once.

CONSTRAINTS

$1 \leq s.length \leq 2000$, and `s` consists of lowercase English letters only.

2 My Intuition

The brute-force idea is to generate every subset of characters (2^n possibilities), store the resulting strings in a set, and read off the size. For `n = 2000` this is completely unfeasible — it blows past both time and memory limits immediately.

So instead of generating subsequences directly, I thought about **counting** them level by level. Let `subs[i]` denote the number of distinct subsequences (including the empty one) that can be formed using the first `i` characters of `s`.

Step 1 — assume all characters are unique

If every character in `s` is distinct, there is no possibility of collisions. Before looking at any character, the only subsequence is the empty string, so `subs[0] = 1`. Every time a new character is introduced, every existing subsequence can either **take** or **not take** that character, so the count exactly doubles:

$$subs[i] = subs[i-1] \times 2$$

Step 2 — account for duplicate characters

Real strings repeat characters, so doubling blindly overcounts. Consider `"xaba"`. Tracking the set of strings explicitly (see the Visual Explanation section) shows that when the second `'a'` appears at index 3, doubling produces the strings `{"", x, a, xa, b, xb, ab, xab, a, xa, aa, xaa, ba, xba, aba, xaba}` — but `{a, xa}` already existed from the first `'a'`, so they must be removed.

The important observation: **a character that has appeared before regenerates exactly as many duplicate strings as the number of subsequences that existed right before its previous occurrence**. That earlier count is precisely `subs[lastSeen[c] - 1]`, where `lastSeen[c]` is the 1-indexed position where character `c` last occurred. This is because every subsequence that existed just before the previous occurrence of `c` gets appended with `c` to form a new subsequence at that time — and doubling from the current occurrence reproduces that exact same set of `"...+c"` strings again.

So a `lastSeen[26]` array is enough to know how many duplicates to subtract, without ever storing actual strings.

3 Approach

- State definition:** `subs[i]` = number of distinct subsequences (empty string included) achievable using the first `i` characters of `s`. Final answer uses `subs[n]`.
- Base case:** `subs[0] = 1` — only the empty subsequence exists before any character is processed.
- Tracking structure:** `lastSeen[26]`, initialized to `-1`, stores the 1-indexed position where each lowercase letter last appeared. This lets us detect repeats in $O(1)$ without storing any strings.
- Transition:** for each index `i` from `1` to `n`, look at character `c = s[i-1]`:
 - Doubling term: `subs[i-1] * 2` — take/not-take the current character against every previously formed subsequence.
 - Duplicate term `dup`: if `c` was never seen before, `dup = 0`. Otherwise `dup = subs[lastSeen[c] - 1]`, the count of subsequences that existed just before `c`'s previous occurrence — these are exactly the strings that get regenerated.
 - `subs[i] = subs[i-1]*2 - dup`, taken modulo 10^9+7 (with a correction to stay non-negative after subtraction).
- Update:** set `lastSeen[c] = i` so future occurrences of the same character know where to look up the duplicate count.
- Answer:** `subs[n]` includes the empty subsequence, so the final answer is `subs[n] - 1` (mod-adjusted).



4 Visual Explanation

Building the subsequence set for `s = "xaba"` step by step (shown explicitly here purely for intuition — the actual algorithm never stores these strings):

i = 0 (before any character) — subs[0] = 1



i = 1 (char 'x') — subs[1] = 2



i = 2 (char 'a') — subs[2] = 4



i = 3 (char 'b') — subs[3] = 8



i = 4 (char 'a' again) — naive doubling gives 16, but 2 are duplicates



subs[4] = 16 − 2 = 14 (struck-through chips are the duplicates removed)

WHERE DOES "2" COME FROM?

The previous `'a'` occurred at index 2, i.e. `lastSeen['a'] = 2`. Just before that occurrence, `subs[lastSeen['a'] - 1] = subs[1] = 2` subsequences existed (`{ "", x }`). Appending `'a'` to those two back then produced `{ a, xa }` — exactly the two strings that reappear now. So `dup = subs[lastSeen['a']-1] = 2`.

lastSeen array evolution

After processing i	char	lastSeen['x']	lastSeen['a']	lastSeen['b']
0	—	-1	-1	-1
1	x	1	-1	-1
2	a	1	2	-1
3	b	1	2	3
4	a	1	4	3

5 Dry Run

Full trace of the algorithm on `s = "xaba"` (1-indexed loop, `s[i-1]` is the character processed at step `i`):

i	s[i-1]	subs[i-1]	lastSeen[c] (before update)	dup	subs[i] = 2*subs[i-1] - dup	lastSeen[c] (after update)
1	x	1	-1	0	2*1 - 0 = 2	1
2	a	2	-1	0	2*2 - 0 = 4	2
3	b	4	-1	0	2*4 - 0 = 8	3
4	a	8	2	subs[2-1] = subs[1] = 2	2*8 - 2 = 14	4

RESULT

`subs[4] = 14` (includes the empty subsequence). Final answer = `subs[4] - 1 = 13` distinct non-empty subsequences of `"xaba"`.

6 Code

```
#include<bits/stdc++.h>
using namespace std;

const int mod = 1e9+7;
int distinctSubseqII(string s) {
    int n = s.size();
```

```
vector<int>lastSeen(26,-1);
vector<int>subs(n+1,0);
subs[0] = 1;
for(int i=1;i<=n;i++){
    int dup = lastSeen[s[i-1]-'a']==-1 ? 0 : subs[lastSeen[s[i-1]-'a']-1];
    subs[i] = (((subs[i-1]<<1)%mod - dup)%mod + mod)%mod;
    lastSeen[s[i-1]-'a'] = i;
}
return (subs[n] - 1 + mod) % mod;
}
```

NOTE (NOT MODIFIED IN THE CODE ABOVE)

`subs[i-1] << 1` left-shifts an `int`. Since `subs[i-1]` is already kept below `mod ≈ 1e9+7`, the shifted value stays under `~2e9`, which just barely fits in a 32-bit signed `int` (max `~2.147e9`). It works here, but it is close enough to the overflow boundary that using `long long` for `subs` (or writing `2*subs[i-1]` with a wider type) would be safer practice in general.

7 Correctness / Why It Works

The core invariant is: `subs[i]` is exactly the number of distinct subsequences obtainable from `s[0..i-1]`, including the empty one. Doubling (`subs[i-1] * 2`) is correct in isolation because pairing "take current char" with every existing subsequence, and "don't take it" (keep every existing subsequence as-is), covers all possibilities exactly once **if** the current character has never appeared before.

When the character **has** appeared before, the "take current char" branch reproduces every subsequence that was formed by appending that same character at its previous occurrence — because appending the same character to the same prefix set yields the same strings. That earlier "take" branch had exactly `subs[lastSeen[c]-1]` subsequences going into it (the state right before that previous occurrence), so that many duplicates are subtracted. This keeps every subsequence counted exactly once, without ever needing to materialize or hash the strings themselves.

8 Complexity

Time	<code>O(n)</code> — a single pass over the string, <code>O(1)</code> work per character (array lookups only, alphabet size is fixed at 26).
Space	<code>O(n)</code> for the <code>subs</code> array, plus <code>O(26)</code> for <code>lastSeen</code> . The <code>subs</code> array can be compressed to <code>O(1)</code> by keeping only the previous value, since each state only depends on <code>subs[i-1]</code> and one earlier value looked up via <code>lastSeen</code> .

9 What I Learned / Key Takeaway

- Counting subsequences directly by generation is exponential; counting **how many exist** via a recurrence avoids ever building the strings.
- The take/not-take doubling pattern is only valid when there's no repetition — repetition needs a correction term, not a different model.
- `lastSeen[c]` combined with `subs[lastSeen[c]-1]` is the trick that converts "which strings get duplicated" into a pure `O(1)` arithmetic lookup.
- Modular subtraction can go negative, so `(... % mod + mod) % mod` is necessary to normalize the result.